

Actividad 3.8. Light & Dark

Básicamente, un tema oscuro muestra principalmente superficies oscuras con menor brillo y contrasta con un tema luminoso dominado por fondos blancos y superficies brillantes.

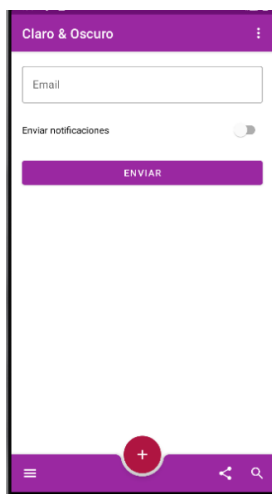
El usuario puede tener la opción de seleccionar un tema claro u oscuro, o que la aplicación tome por defecto el tema que tenga seleccionado el usuario en el sistema operativo.

Obviando discusiones sobre si es mejor uno u otro estilo, nosotros ahora vamos a crear una pequeña App donde trabajaremos y diseñaremos ambos estilos y donde el usuario podrá cambiar de uno a otro según prefiera. Crea un proyecto y llámale "Light&Dark":

1.	Diseño de los 2 layout de la App y la PreferenceScreen/Fragmento	1
1.1.	Layout/Ventana principal "activity_main"	2
1.2.	Layout/Ventana de "Settings". "activity_settings"	6
2.	Tema claro y oscuro de la app.....	11
2.1.	Colores.....	11
2.2.	Temas.....	12
3.	AndroidManifest.xml	14
4.	Lógica (archivos kotlin del proyecto)	14

1. Diseño de los 2 layout de la App y la PreferenceScreen/Fragmento

La aplicación constará de una layout principal, uno segundo layout para acceder al menú de seleccionar tema (que se compone de otros 3 archivos xml):



activity_main.xml

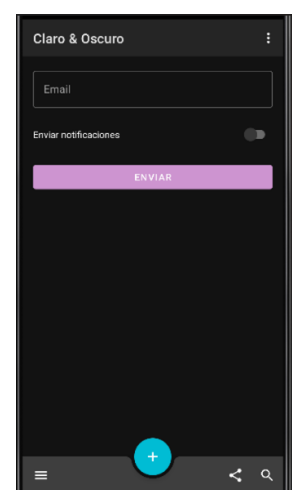
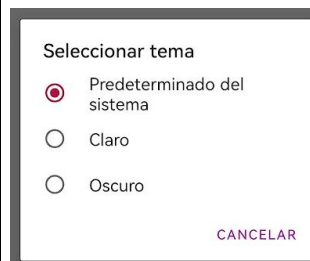
toolbar_menu.xml

bottom_app_bar_menu.xml



activity_settings.xml

menu_settings.xml (fragmento)

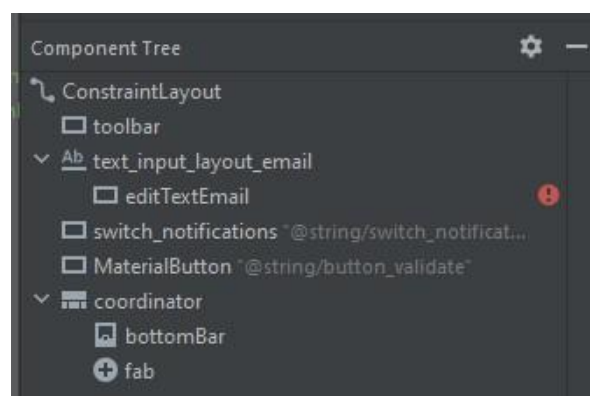


Modo oscuro

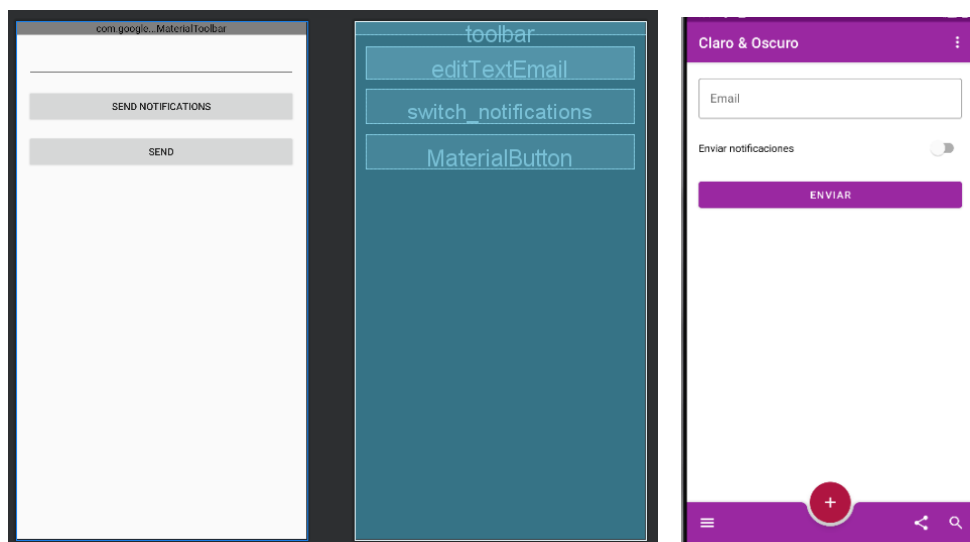
1.2. LAYOUT/VENTANA PRINCIPAL “ACTIVITY_MAIN”

a) La “activity_main”, será un **ConstraintLayout** que contiene estos componentes:

- Una **MaterialToolbar** que actúa de barra de la aplicación (action bar o app bar). Incluye un menú desplegable.
- Un campo de entrada **TextInputLayout** para escribir una dirección de correo electrónico.
- Un botón que verifica el formato del texto introducido en el **TextInputLayout**.
- Un **conmutador de opción (Switch)**.
- Una barra inferior de tipo **BottomAppBar** con un botón flotante (FAB o Floating Action Button) incrustado.



El resultado final será:



Fíjate que es un **ConstraintLayout**, el cual tiene 2 layout dentro de él (un **TextInputLayout**, y un **CoordinatorLayout**)

Ahora vete insertando los elementos **según estos parámetros principales** (tienes que ir completando tú el resto de atributos (“Android:layout_xxxx”, “app:layout_constraintxxxxx”, “android:hint” ...)).

NOTA 1: Los iconos tienes que descargarlos de la actividad del Moodle (zip) y déjalos en la carpeta "drawable". Revisalos para ver cómo son.

NOTA 2: Descarga también el archivo strings.xml y déjalo en la carpeta values. Revisa el archivo para ver su contenido.

Elementos del ConstraintLayout:

MaterialToolbar, es un componente de la biblioteca de diseño de materiales de Android (com.google.android.material) que representa la barra de herramientas (Toolbar) pero con estilos específicos y mejoras proporcionadas por Material Design:

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

TextInputLayout se utiliza para proporcionar una envoltura alrededor de un campo de entrada de texto (EditText), brindando algunas funcionalidades adicionales y estilos específicos de Material Design:

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/text_input_layout_email"
```

TextInputEditText es una subclase de **EditText** proporcionada por Material Design:

```
<com.google.android.material.textfield.TextInputEditText
    android:id="@+id/editTextEmail"
```

SwitchMaterial: proporciona una implementación mejorada y estilizada del interruptor (switch) en comparación con la implementación estándar. Este componente que utilizamos pertenece al conjunto de componentes de Material Design de Google:

```
<com.google.android.material.switchmaterial.SwitchMaterial
    android:id="@+id/switch_notifications"
```

MaterialButton es un componente de la biblioteca el cual representa un botón con estilos específicos y mejoras proporcionadas por Material Design. Además, se le ha añadido un "onClick":

```
<com.google.android.material.button.MaterialButton
    android:onClick="validate"
    android:text="Send"
```

Un **CoordinatorLayout**, que es un contenedor de diseño ampliamente utilizado en el desarrollo de aplicaciones Android para facilitar la coordinación de vistas y comportamientos complejos:

```
<androidx.coordinatorlayout.widget.CoordinatorLayout  
    android:id="@+id/coordinator">
```

AppBar es un componente que representa una barra de aplicaciones en la parte inferior de la pantalla:

```
<com.google.android.material.bottomappbar.BottomAppBar  
    android:id="@+id/bottomBar">
```

FloatingActionButton. Este componente representa un botón flotante que generalmente se utiliza para acciones principales o destacadas en nuestra app:



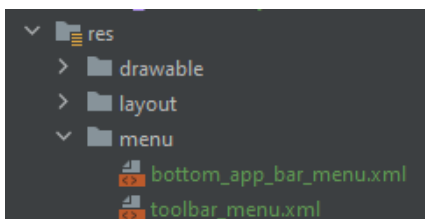
```
<com.google.android.material.floatingactionbutton.FloatingActionButton  
    android:id="@+id/fab"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_anchor="@id/bottomBar"  
    app:srcCompat="@drawable/ic_add" />
```

`app:layout_anchor="@id/bottomBar"` <!-- Identificador de la vista a la que el FAB estará anclado -->

- b) Ahora también vamos a crear la **ventana superior** que sale al pulsar los “3 puntos” de la barra de menú:



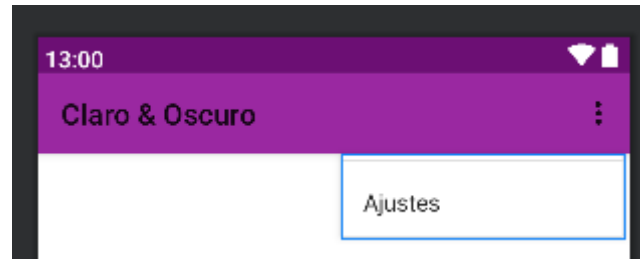
Primero crea una **carpeta** dentro de “res” y llámala, “menu”, y dentro de ella crea el archivo “**toolbar_menu**”, donde insertaremos la barra superior:



```
<menu xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto">  
  
    <item  
        android:title="@string/toolbar_action_settings"  
        app:showAsAction="never" />  
  
</menu>
```

toolbar_menu.xml

app:showAsAction="never": Especifica cómo se debe mostrar este elemento en la barra de herramientas. En este caso, se establece en "never", lo que significa que este elemento no debe mostrarse como un icono en la barra de herramientas y se mostrará en el menú desplegable cuando el usuario toque el botón de menú en la barra de herramientas:



toolbar_menu.xml

- c) Y lo mismo para los botones que aparecen en la barra de debajo de la ventana principal. Crea el **archivo bottom_app_bar_menu.xml**, en la misma carpeta que el punto b) e inserta los 2 iconos:



```
3 <menu xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto">
5
6     <item
7         android:id="@+id/bottom_app_bar_menu_share"
8         android:title="Share"
9         android:icon="@drawable/ic_share"
10        app:showAsAction="always"/>
11
12    <item
13        android:id="@+id/bottom_app_bar_menu_search"
14        android:title="Search"
15        android:icon="@drawable/ic_search"
16        app:showAsAction="always"/>
17 </menu>
```

bottom_app_bar_menu

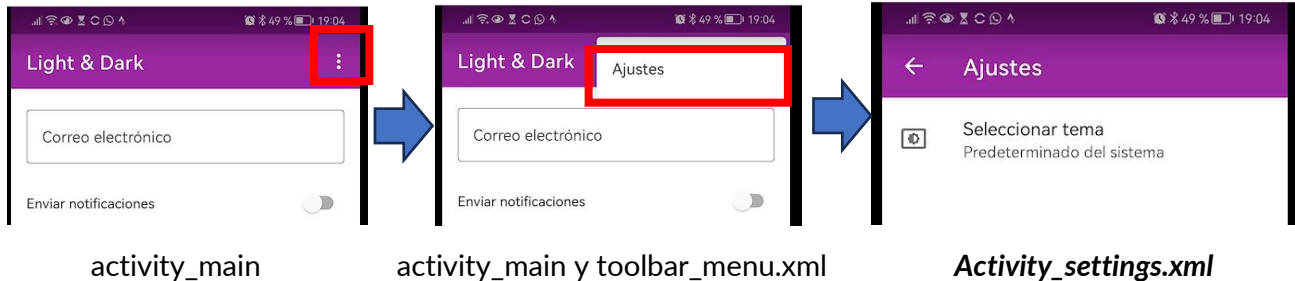
d) Dimensiones App → **dimens.xml**

Para facilitar la gestión, la coherencia en la apariencia de la interfaz, y ajustar fácilmente estas dimensiones desde un solo archivo, vamos a crear el archivo **"res/values/dimens.xml"**:

```
<resources>
    <!-- Márgenes de pantalla predeterminados -->
    <dimen name="activity_horizontal_margin">16dp</dimen>
    <dimen name="activity_vertical_margin">16dp</dimen>
</resources>
```

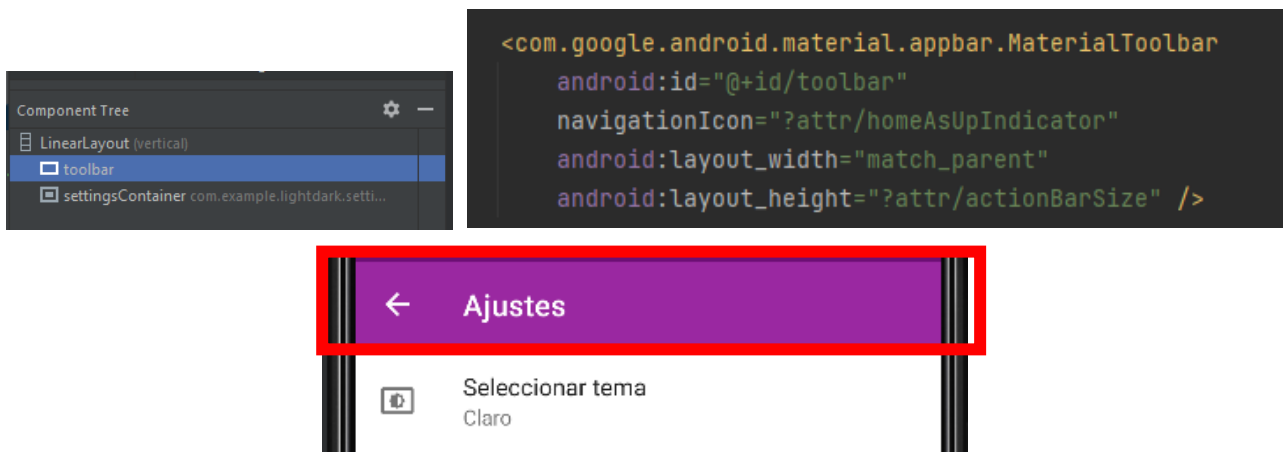
1.3. LAYOUT/VENTANA DE "SETTINGS". "ACTIVITY_SETTINGS"

Desde la layout principal, a través de un botón iremos a esta segunda layout donde aparecerá el menú para seleccionar el tema que queremos elegir:



Este nuevo layout "Activity_settings.xml" será un "LinearLayout" con una "MaterialToolbar" y un "FragmentManager" (settingsContainer) ambos en el grupo Containers de la ventana diseño "Palette":

a) Una toolbar:



b) Un settingscontainer:

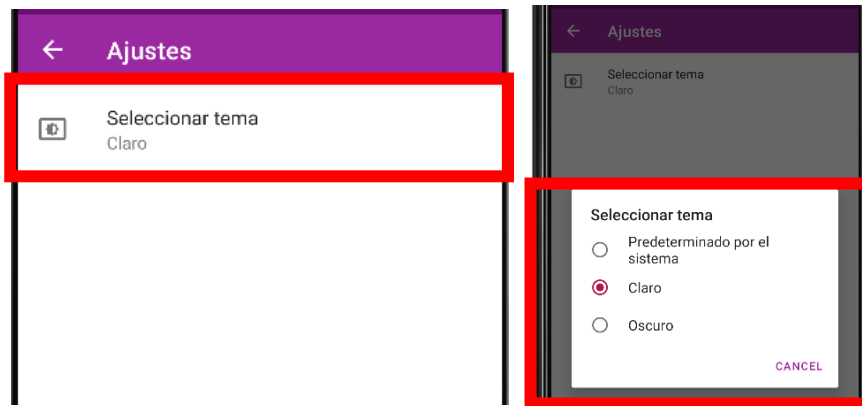
El contenedor del fragmento es un "FragmentManager" que es una vista personalizada que deriva de FrameLayout y se utiliza como contenedor recomendado para fragmentos. A diferencia de otros ViewGroups (FrameLayout, LinearLayout, ...), FragmentManager solo acepta vistas de fragmentos.

```
<androidx.fragment.app.FragmentManager
    android:id="@+id/settingsContainer"
    android:tag="settings_fragment_tab"
    android:name="com.example.lightdark.settings.SettingsFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Donde vemos que el nombre del contenido del fragmento será:
"com.example.lightdark.sttings.SettingsFragment", el cual **tenemos que crear**.

Este código configura el **fragmento** para la **selección de los temas (Predeterminado/Claro/Oscuro)**, que está diseñado a través de un archivo XML (**menu_settings.xml**), que tienes que crear y ubicarlo en la ruta **“res\xml\menú_settings.xml”**.

Además, incluirá el manejo específico para el **cambio de tema** utilizando la clase **ThemeSetup**. El diseño de la ventana que vas a diseñar ahora aparecerá así en la app:

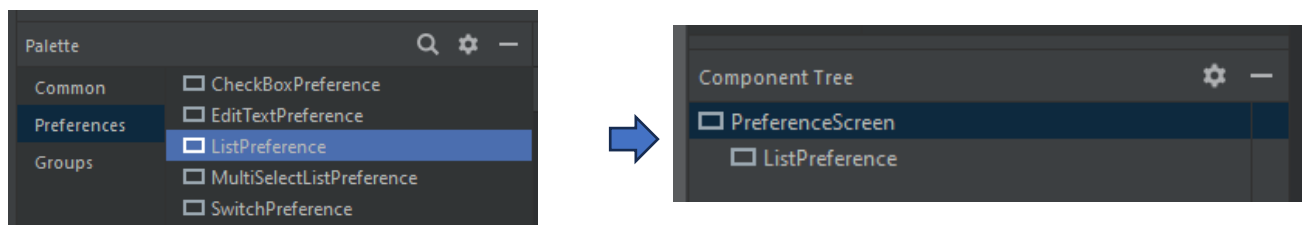


menu_settings.xml

Es una lista de selección, denominada **“ListPreference”**, que tienes que crear en un archivo xml del tipo **“XML Resource File”** dentro la carpeta **“res\values\xml”**, y le vas a llamar **“menu_settings”**.

c) menu_settings.xml

Abriendo el archivo, fíjate que el tipo de ventana es una **“PreferenceScreen”** y dentro de ella el contenido que tienes que introducir es una **“ListPreference”** (presenta una lista de opciones en un cuadro de diálogo. De todas ellas, el usuario elegirá una, si así lo desea):

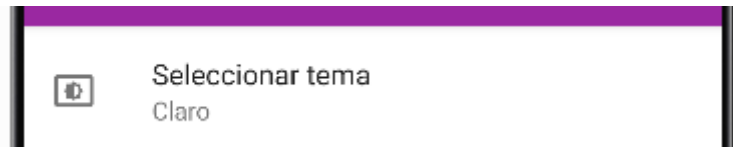


```
<androidx.preference.PreferenceScreen
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <androidx.preference.ListPreference
        android:key="@string/settings_theme_key"
        android:title="@string/settings_theme_title"
        android:icon="@drawable/baseline_settings_brightness_24"
        android:dialogTitle="@string/settings_theme_dialog_title"
        app:useSimpleSummaryProvider="true"
        android:entries="@array/settings_theme_entries"
        android:entryValues="@array/settings_theme_values"
        android:defaultValue="@string/settings_theme_value_default"/>

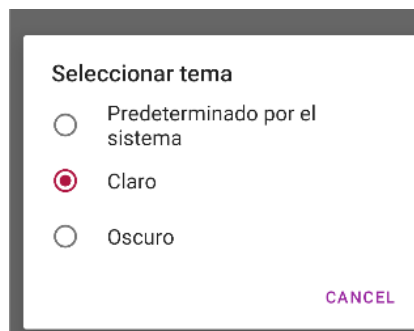
</androidx.preference.PreferenceScreen>
```


- **android:key**. El nombre identificativo de la lista gestionada por el componente
- **android:title**. El título de la lista. Es el texto que verá el usuario ("Seleccionar tema").
- **android:icon**. El icono que simboliza la preferencia. Un recurso drawable.
- **android:dialogTitle**. El título del cuadro de diálogo con la lista de opciones asignables a theme.
- **app:useSimpleSummaryProvide**. Si es true, el componente muestra el valor de la lista.



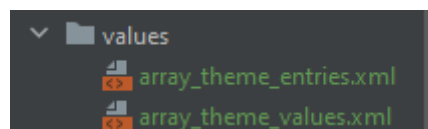
Ahora veremos los atributos de ListPreference relacionados con los posibles valores del TEMA; es decir, las opciones de configuración (la app de ejemplo ofrecerá tres: tema claro, tema oscuro y el predeterminado)

- **android:entries="@array/settings_theme_entries"** → El nombre del fichero con los nombres de las opciones seleccionables en el listado (los temas)
- **android:entryValues="@array/settings_theme_values"** → El nombre del fichero que contiene los valores del tema que elegiremos.
- **android:defaultValue="@string/settings_theme_value_default"** → El valor predeterminado de theme. Ten en cuenta que el valor de una preferencia será nulo hasta que el usuario escoja el valor por primera vez. Mientras sea nulo, la pantalla de preferencias indicará que el valor "Predeterminado" es el seleccionado.



Por ejemplo (como es el caso), si settings_theme_value_default contiene la cadena "Predeterminado", entonces el tema predeterminado para la preferencia será "**Predeterminado**" hasta que el usuario cambie explícitamente esta configuración.

Estos campos (@array) los vamos a obtener de **2 archivos independientes**, Por lo tanto tendremos que crear ahora esos 2 archivos dentro la carpeta "**res/values**"



array_theme_entries.xml (fíjate que el **name** es el que buscamos de la "ListPreference", "entries")

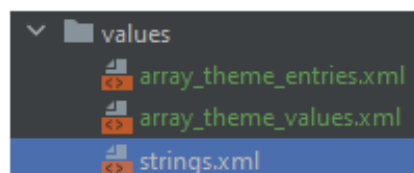

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="settings_theme_entries">
        <item>Predeterminado del sistema</item>
        <item>Claro</item>
        <item>Oscuro</item>
    </string-array>
</resources>
```

array_theme_values.xml (fíjate que el **name** es el que buscamos de la “ListPreference”, entryValues)

Estos valores que elegirá el usuario, los vamos a declarar como constantes recuperables tanto en XML como en el código en el archivo strings.xml (el que has descargado y añadido antes):

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="settings_theme_values">
        <item>@string/settings_theme_value_default</item>
        <item>@string/settings_theme_value_light</item>
        <item>@string/settings_theme_value_dark</item>
    </string-array>
</resources>
```

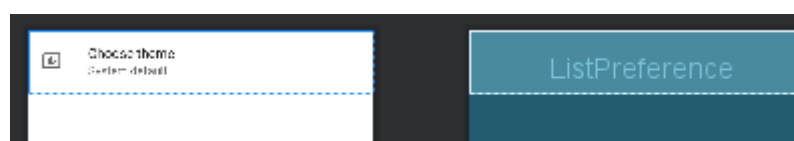
NOTA: Si no lo has descargado todavía, coge el archivo **“strings.xml” de Moodle** y cárgalo en tu proyecto en la carpeta **“values”**. Revisa su contenido.



donde tendrías que colocar el siguiente código (ya debería de estar añadido):

```
<string name="settings_theme_value_default" translatable="false">DEFAULT</string>
<string name="settings_theme_value_light" translatable="false">LIGHT</string>
<string name="settings_theme_value_dark" translatable="false">DARK</string>
```

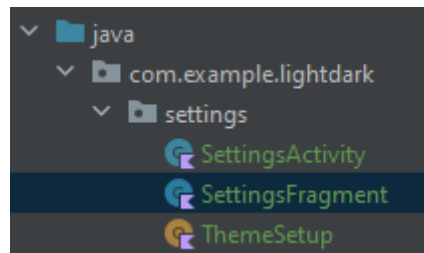
En el modo diseño solo podemos visualizar este aspecto:



La lógica para la ejecución de este fragmento se realiza a través del archivo “SettingsFragment.kt” que has creado antes.

d) Lógica del fragmento:

Primero crea una carpeta dentro de la parte “lógica/kotlin” del proyecto y llámale “settings”, y dentro de ella, crearemos varios archivos, entre ellos, el que nos interesa ahora, “SettingsFragment”:



El código kotlin para habilitarlo es:

```
class SettingsFragment : PreferenceFragmentCompat() {  
    override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {  
        setPreferencesFromResource(R.xml.menu_settings, rootKey)  
    }  
}
```

Te dará error “PreferenceFragmentCompat” y es porque tienes que añadir al final del archivo build.gradle.kts:

```
implementation("androidx.preference:preference-ktx:1.2.1")
```

Y en el SettingsFragment.kt que estás definiendo en esta página las importaciones:

```
package com.example.lightdark.settings  
  
import android.os.Bundle  
import androidx.preference.Preference  
import androidx.preference.PreferenceFragmentCompat  
import com.example.lightdark.R
```

Esto resultaría suficiente para que el usuario vea la pantalla, abra el cuadro de diálogo pulsando la preferencia, seleccione una opción (el valor para el tema) y esta se guarde de manera automática.

Pero necesitamos también saber qué tema es el que está guardado como predeterminado o si el usuario ha cambiado ese valor por otro (Claro u Oscuro), para indicarlo al entrar en la ventana. Se llama a la clase “**ThemeSetup**” que se crea en el archivo “ThemeSetup.kt” (está más adelante):

```
findPreference<Preference>(getString(R.string.settings_theme_key))?.setOnPreferenceChangeListener { preference, newValue ->
    if (newValue is String) {
        ThemeSetup.applyTheme(newValue, requireContext())
        true ^setOnPreferenceChangeListener
    } else {
        false ^setOnPreferenceChangeListener
    }
}
```

2. Tema claro y oscuro de la app

1.4. COLORES

Como sabemos, los colores se definen en el fichero `/res/values/colors.xml`, en este caso vamos a utilizar 9 colores para toda la app, y los obtenemos de las paletas:

	50	100	200	300	400	500	600	700	800	900
Red										
Pink										
Purple										

El valor que va de 50 a 900, define la intensidad del color, y utilizaremos ese criterio para el **nombre** del color que vayamos a utilizar:

```
3 <resources>
4 <color name="purple_500" type="color">#9a28a1</color>
5 <color name="purple_900" type="color">#6c0f74</color>
6 <color name="purple_300" type="color">#cd94d0</color>
7
8 <color name="red_500" type="color">#af1541</color>
9 <color name="red_300" type="color">#c75b7a</color>
10
11 <color name="white_50">#ffffff</color>
12
13 <color name="black_900">#000000</color>
14
15 <color name="grey_900">#252525</color>
16
17 <color name="cyan_500">#00bcd4</color>
18
19 </resources>
```

colors.xml

Como vimos en actividades anteriores, puedes seleccionar los colores base en herramientas gratuitas como Figma, [Material Design](#) o [Adobe color](#).

1.5. TEMAS

Como podremos seleccionar **3 temas** (“Predeterminado del sistema”, “Claro” y “Oscuro”), lo que vamos a hacer es utilizar 1 tema para los 2 primeros (**res\values\themes.xml**) y un segundo tema para el 3º (**res\values-night\themes.xml**).

De **manera genérica**, la lista de temas de componentes de materiales que se podrían utilizar para obtener los últimos estilos de componentes y atributos a nivel de tema en “Material Design 2” serían:

- `Theme.MaterialComponents`
- `Theme.MaterialComponents.NoActionBar`
- `Theme.MaterialComponents.Light`
- `Theme.MaterialComponents.Light.NoActionBar`
- `Theme.MaterialComponents.Light.DarkActionBar`
- `Theme.MaterialComponents.DayNight`
- `Theme.MaterialComponents.DayNight.NoActionBar`
- `Theme.MaterialComponents.DayNight.DarkActionBar`

<https://m2.material.io/develop/android/docs/getting-started>

- A) Establecemos los colores para el tema **Claro/Oscuro “Predeterminado del sistema”**.
En este caso se va a utilizar “**Theme.MaterialComponents.Light.NoActionBar**”.

El nombre del estilo (**style name**) define un tema llamado **AppTheme.LightDark**. El resto del fichero sobrescribe algunas de los numerosos ítems (colores, estilos de componentes, etcétera) de **Theme.MaterialComponents.Light.NoActionBar**. Dado que el tema padre ya provee un diseño visual completo, sobrescribe solo aquellos ítems que quieras personalizar:

```
<!-- Tema base para el Claro-Oscuro (Predeterminado por el sistema) -->

<style name="Base.AppTheme.LightDark" parent="Theme.MaterialComponents.DayNight.NoActionBar">
    <item name="android:forceDarkAllowed" tools:targetApi="q">false</item>
    <item name="colorOnPrimary">@color/white_50</item>
    <item name="colorOnSecondary">@color/white_50</item>
    <item name="colorError">@color/red_300</item>
    <item name="bottomAppBarStyle">@style/Widget.MaterialComponents.BottomAppBar.PrimarySurface</item>

    <!-- toolbar -->

    <item name="toolbarStyle">@style/Widget.MaterialComponents.Toolbar.PrimarySurface</item>
    <item name="actionOverflowButtonStyle">@style/ToolbarStyle.Overflow</item>
    <item name="toolbarNavigationButtonStyle">@style/Toolbar.Button.Navigation.Tinted</item>

    <!-- menus -->

    <item name="popupTheme">@style/Theme.MaterialComponents.DayNight</item>
</style>

<style name="ToolbarStyle.Overflow" parent="Widget.AppCompat.ActionButton.Overflow">
    <item name="android:tint">@color/white_50</item>
</style>

<style name="Toolbar.Button.Navigation.Tinted" parent="Widget.AppCompat.Toolbar.Button.Navigation">
    <item name="tint">@color/white_50</item>
</style>
```

res\values\themes.xml (2)

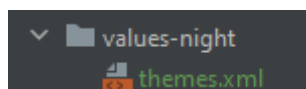
- B) Para el **tema claro**, vamos a crear un estilo personalizado para solo establecer los colores primarios y secundarios que vamos a necesitar:

```
<!-- Tema claro de la App -->

<style name="AppTheme.LightDark" parent="Base.AppTheme.LightDark">
    <item name="colorPrimary">@color/purple_500</item>
    <item name="colorPrimaryVariant">@color/purple_900</item>
    <item name="colorSecondary">@color/red_500</item>
    <item name="android:statusBarColor">?attr/colorPrimaryVariant</item>
</style>
```

res\values\themes.xml (1)

- C) Ahora hay que crear el **tema para el modo oscuro** en el archivo **res\values-night\themes.xml**:



```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <!-- App theme (dark version)-->
    <style name="AppTheme.LightDark" parent="Base.AppTheme.LightDark">
        <item name="colorPrimary">@color/purple_300</item>
        <item name="colorPrimaryVariant">@color/purple_500</item>
        <item name="colorSecondary">@color/cyan_500</item>
        <item name="colorSurface">@color/grey_900</item>
        <item name="android:statusBarColor">@color/black_900</item>
    </style>
</resources>
```

res\values-night\themes.xml

3. AndroidManifest.xml

En este caso, en este archivo tendremos que tener en cuenta las 2 activities que tenemos creadas:

```
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

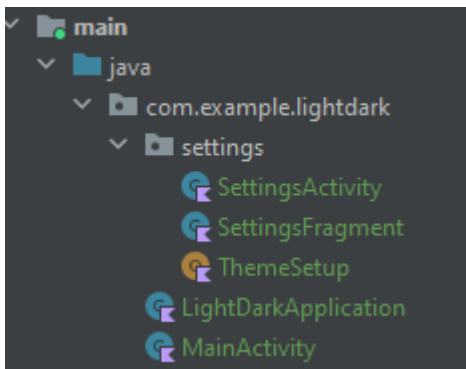
MainActivity

```
<activity
    android:name=".settings.SettingsActivity"
    android:label="Settings">
    <meta-data
        android:name="android.support.PARENT_ACTIVITY"
        android:value=".MainActivity" />
</activity>
```

SettingsActivity

4. Lógica (archivos kotlin del proyecto)

Los archivos que necesitamos para que la app funcione se compone de los siguientes archivos:



Para tenerlo organizado, los 3 archivos que necesitamos y que hacen referencia al cambio de tema claro y oscuro, están dentro la carpeta “settings”

- A) SettingsFragment es el que has creado antes.
- B) SettingsActivity (de “activity_settings.xml”)

```
new *
class SettingsActivity : AppCompatActivity() {
    new *
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_settings)
        setSupportActionBar(findViewById(R.id.toolbar))
        supportActionBar!!.setDisplayHomeAsUpEnabled(true)
    }

    new *
    companion object {
        new *
        fun start(context: Context) {
            val intent = Intent(context, SettingsActivity::class.java)
            context.startActivity(intent)
        }
    }
}
```

- C) ThemeSetup (para gestionar el tema seleccionado por el usuario como predeterminado)


```
new *
object ThemeSetup {
    new *
    fun applyTheme(mode: String?, context: Context) {
        if ("DARK" == mode) {
            AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES)
        } else if ("LIGHT" == mode) {
            AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO)
        } else {
            AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_FOLLOW_SYSTEM)
        }
    }
}

new *
fun applyTheme(context: Context) {
    val defaultSharedPreferences = PreferenceManager.getDefaultSharedPreferences(context)
    val value = defaultSharedPreferences.getString(
        "theme",
        "DEFAULT"
    )
    applyTheme(value, context)
}
```

D) LightDarkApplication (para el inicio de la aplicación)

Cada vez que la app arranque debería establecer el modo que el usuario haya seleccionado en la pantalla de preferencias, si en algún momento eligió alguno. Y esta operación hay que escribirla.

Ejecutar código durante el arranque de una aplicación en una especialización de la **clase Application**. Su método `onCreate()` se invoca antes de la creación de cualquier actividad. En él accedemos al contexto global de Android con el objeto **this**.

```
package com.example.lightdark

import ...

new *
class LightDarkApplication : Application() {
    new *
    override fun onCreate() {
        super.onCreate()
        ThemeSetup.applyTheme(this)
    }
}
```

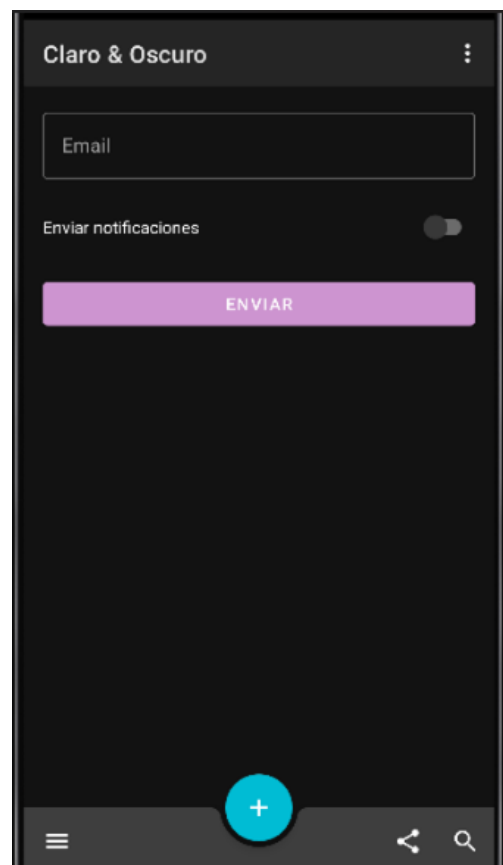
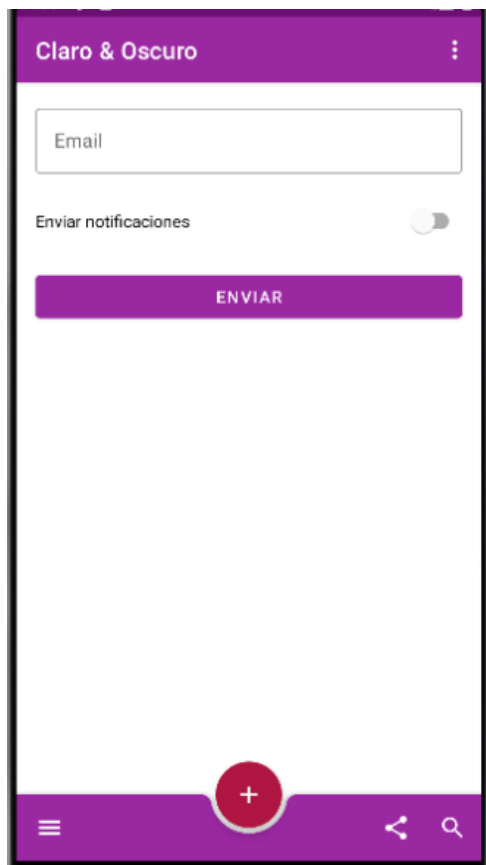
E) MainActivity (donde va la lógica de la activity_main (error al no poner email,) :

```
1 package com.example.lightdark
2
3 import android.content.Context
4 import android.os.Bundle
5 import android.text.TextUtils
6 import android.view.Menu
7 import android.view.MenuItem
8 import android.view.View
9 import android.view.inputmethod.InputMethodManager
10 import android.widget.EditText
11 import androidx.appcompat.app.AppCompatActivity
12 import androidx.appcompat.widget.Toolbar
13 import com.example.lightdark.settings.SettingsActivity
14 import com.google.android.material.textfield.TextInputEditText
15 import com.google.android.material.textfield.TextInputLayout
```

```
new *
17 class MainActivity : AppCompatActivity() {
18
19     private lateinit var editTextEmail: TextInputEditText
20     private lateinit var textInputEmail: TextInputLayout
21
22     new *
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25         setContentView(R.layout.activity_main)
26
27         val toolbar: Toolbar = findViewById(R.id.toolbar)
28         setSupportActionBar(toolbar)
29
30         editTextEmail = findViewById(R.id.editTextEmail)
31         textInputEmail = findViewById(R.id.text_input_layout_email)
32     }
33
34     new *
35     override fun onCreateOptionsMenu(menu: Menu): Boolean {
36         menuInflater.inflate(R.menu.toolbar_menu, menu)
37         return super.onCreateOptionsMenu(menu)
38     }
39 }
```

```
override fun onCreateOptionsMenu(menu: Menu): Boolean {  
    menuInflater.inflate(R.menu.toolbar_menu, menu)  
    return super.onCreateOptionsMenu(menu)  
}  
  
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
    SettingsActivity.start(this)  
    return true  
}  
  
fun validate(view: View) {  
    if (TextUtils.isEmpty(editTextEmail.text)) {  
        textInputEmail.error = "Este campo es obligatorio"  
        textInputEmail.isErrorEnabled = true  
    } else {  
        textInputEmail.error = null  
        textInputEmail.isErrorEnabled = false  
    }  
}  
}
```

Una vez completado todo, compila el proyecto para ver errores y prueba a visualizarlo en un dispositivo virtual, el resultado debería ser algo similar a esto:



Crea el archivo apk, el proyecto comprimido en zip y súbelos a Moodle.